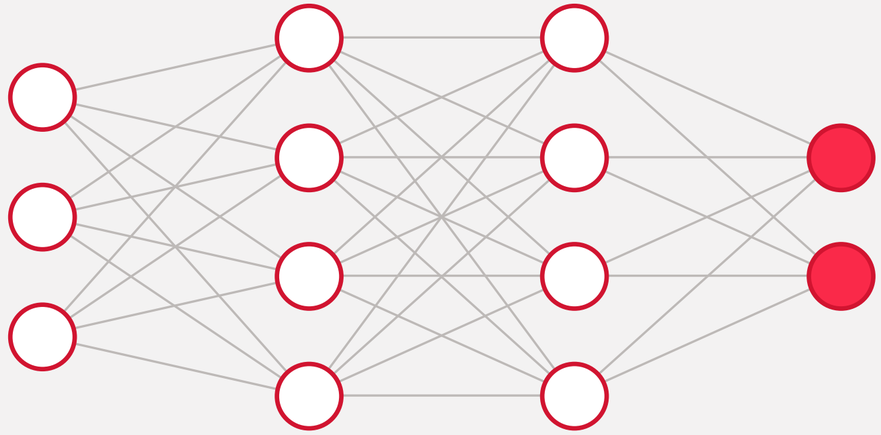
Fundamentos de IA y Python

Unidad 1: Conceptos, herramientas y bases de Python para el curso.



Contenido

01 Introducción a la IA, herramientas y entorno

02 Trabajando con notebooks

03 Python básico

04 Python intermedio

05 Programación asíncrona

06 Validación de datos con Pydantic

01

Introducción a la IA, Herramientas y Entorno

01

¿Qué es la Inteligencia Artificial?

A lo largo de la historia se ha definido la IA desde perspectivas distintas, lo que ha generado enfoques distintos en su desarrollo.

Simular inteligencia

“La IA es la ciencia y la ingeniería de hacer máquinas inteligentes”

— McCarthy, 1956

vs.

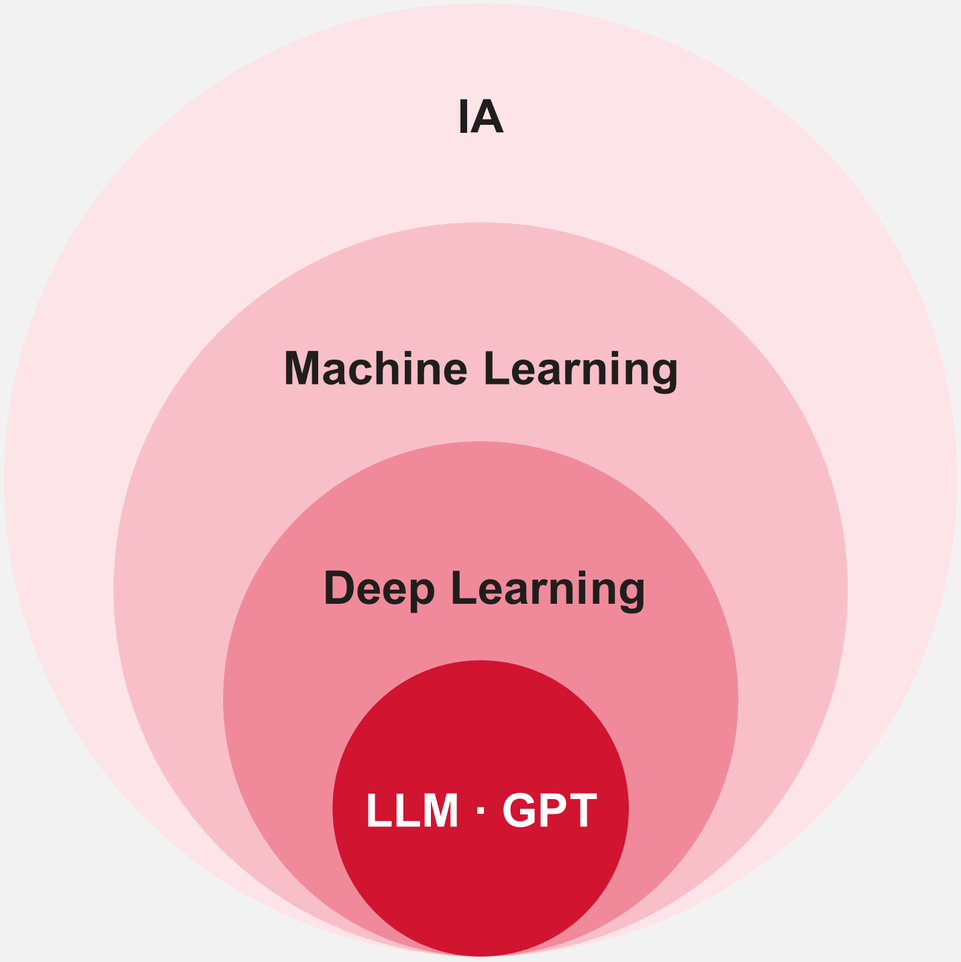
Actuar racionalmente

“La IA es el estudio de agentes que perciben su entorno y toman acciones para maximizar su probabilidad de éxito”

— Russell & Norvig, 2021

No hay una única definición "correcta", cada una enfatiza algo distinto: simular inteligencia vs. actuar racionalmente en un entorno.

Tipos de Inteligencia Artificial



IA

Simula procesos cognitivos humanos: aprendizaje, razonamiento, decisión

Machine Learning

Aprende de datos y mejora su rendimiento sin ser programado explícitamente

Deep Learning

Usa redes neuronales profundas para modelar patrones complejos

LLM

Modelos entrenados con grandes cantidades de texto para lenguaje natural

GPT

Modelos que generan texto mediante redes neuronales entrenadas en texto

IA Generativa

Crea contenido nuevo (texto, imágenes, música) a partir de patrones aprendidos

Lenguajes de programación para IA

Python

Versátil, fácil de aprender.

Librerías: TensorFlow, PyTorch, scikit-learn

Lenguaje del curso

R

Análisis estadístico y visualización

Usos:
Ciencia de datos

Java

Robusto y portable.

Usos:
Aplicaciones empresariales de IA, Weka

C++

Rápido y eficiente.

Usos:
Sistemas de IA de alto rendimiento y juegos

Este curso trabaja íntegramente en Python, el lenguaje dominante en IA y ciencia de datos.

Frameworks y entornos de desarrollo

Frameworks

Librerías para construir modelos

TensorFlow	De Google, flexible para redes neuronales y deep learning
PyTorch	Popular por su facilidad de uso e ideal para investigación
scikit-learn	Biblioteca de Python para ML clásico, simple de usar
Keras	API de alto nivel sobre TensorFlow

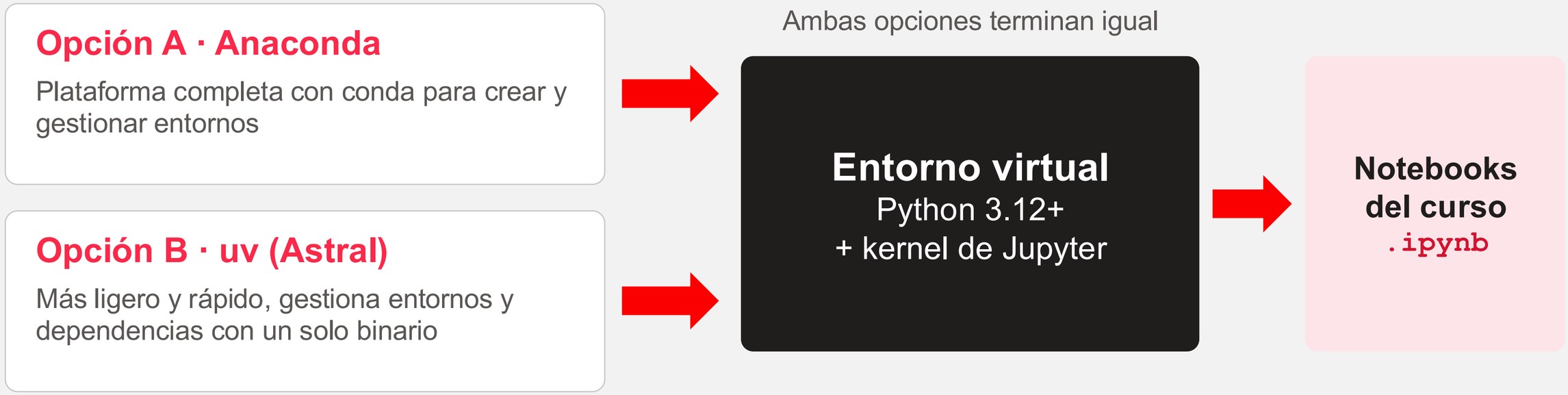
Entornos

Dónde se escribe y ejecuta el código

Anaconda	Gestión de entornos y paquetes de Python
uv (Astral)	Alternativa más ligera y rápida a Anaconda/conda
VS Code	Editor ligero con personalización mediante extensiones
Jupyter	Entorno interactivo para ciencia de datos
Google Colab	Notebooks en la nube, sin instalación local

Instalación del entorno de trabajo

El curso usa Python 3.12+ dentro de un entorno virtual (aislado del resto del sistema).



Los comandos exactos (conda/uv, por sistema operativo) están en el notebook.

02

Trabajando con Notebooks

02

¿Qué es un notebook y cómo se estructura?

Un notebook combina código ejecutable, texto explicativo y resultados en un mismo documento. Se usa en VS Code, Google Colab o Jupyter Notebook, seleccionando el entorno (kernel) que se creó en la Unidad 1.

- Celdas de código: se ejecutan con Shift + Enter
- Celdas de markdown: texto, títulos e imágenes

El resultado de la última instrucción de una celda se muestra automáticamente debajo de ella

Markdown

Análisis de notas

Texto explicativo con formato

Código · Shift + Enter

```
In [1]:  promedio = (8 + 9) / 2
         promedio
```

Resultado

```
Out [1]:  8.5
```

El orden de ejecución importa

Las celdas no tienen que ejecutarse en el orden en que aparecen en la página, y eso puede generar resultados inesperados.

- La primera vez imprime "bananas"
- Si se reejecuta la celda de impresión sin correr las anteriores, el resultado cambia
- El notebook recuerda el estado de las variables, no el orden visual

```
fruta_favorita = "bananas"
# ... más abajo en el notebook ...
fruta_favorita = f"cualquier cosa menos {fruta_favorita}"

# celda de impresión, ejecutada dos veces:
print(f"Mi fruta favorita es {fruta_favorita}")
```

Lo que importa es la memoria del kernel



print muestra el valor que haya en memoria en ese momento, no el que se ve arriba en la página.

El kernel

- El kernel es el proceso de Python que ejecuta el código del notebook y guarda su estado (variables, imports, etc)
- Reiniciar el kernel borra todas las variables definidas. Hay que volver a ejecutar las celdas para asignar nuevamente sus valores
- Un kernel "colgado" o con estado confuso se soluciona casi siempre reiniciándolo



Herramientas complementarias

`%ls  %pwd`

Comandos mágicos de IPython: funcionan igual en Windows, macOS y Linux

`!pip install`

Comandos de shell con !, para correr instrucciones del sistema

`tqdm`

Agrega una barra de progreso a un bucle largo

`Markdown`

IPython.display.Markdown muestra texto con formato enriquecido como salida de una celda

```python
from tqdm import tqdm

for i in tqdm(range(100)):
    ...
# barra de progreso
# en vez de esperar a ciegas
```

Salida en el notebook



`70/100`

Es visible el avance en lugar de esperar a ciegas.

03

Python Básico

Variables y tipos de datos

- Python no exige declarar el tipo de variable, se infiere del valor asignado
- str, int, float, bool son los tipos básicos más usados
- type(variable) muestra el tipo real de cualquier valor

```
nombre = "Ana"            # str (texto)
edad = 21                 # int (entero)
promedio = 8.7            # float (decimal)
es_estudiante = True      # bool (verdadero/falso)

print(f"{nombre} tiene {edad} años")
```

str

texto

`"Ana"`

int

entero

`21`

float

decimal

`8.7`

bool

verdadero / falso

`True`

Estructuras de datos

Lista `(list)`




```
["manzana", "banana"]
```

Ordenada y modificable

Tupla `(tuple)`

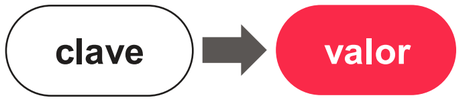


valores fijos

```
(4.6, -74.0)
```

Ordenada e inmutable

Diccionario `(dict)`




```
{"nombre": "Carlos"}
```

Pares clave-valor

Conjunto `(set)`




```
{"Python", "R"}
```

Sin duplicados, sin orden

Elegir la estructura correcta simplifica el resto del código: listas para secuencias, dict para búsquedas por clave.

Control de flujo

if

if / elif / else

Ejecuta un bloque según una condición

for

for

Recorre los elementos de una colección uno por uno

while

while

Repite mientras una condición se mantenga verdadera

```
if temperatura > 25:
    print("Hace calor")
elif temperatura > 15:
    print("Clima templado")
else:
    print("Hace frío")

for fruta in frutas:
    print(fruta)

while contador < 5:
    contador += 1
```

Resultado según la temperatura

≤ 15	16 – 25	> 25
Hace frío	**Templado**	**Hace calor**

Excepciones y funciones

try / except

Evita que un error detenga todo el programa. Maneja el caso de excepción esperado



Funciones (def)

Se definen una vez y se reutilizan con distintos argumentos




```python
try:
    resultado = 10 / 0
except ZeroDivisionError:
    print("No se puede dividir entre cero")

def saludar(nombre_persona):
    return f"¡Hola, {nombre_persona}!"
```

Resultado

```
No se puede dividir entre cero
>>> saludar("Ana")
'¡Hola, Ana!'
```

04

Python Intermedio

04

Comprensiones de listas y diccionarios

- Una comprensión construye una lista o diccionario nuevo en una sola línea, a partir de otro iterable
- Se puede filtrar con if dentro de la misma expresión
- Más compacto y, para la mayoría, más legible que el bucle equivalente

```
fruits = ["Apples", "Bananas", "Pears"]

# lista nueva, sin bucle for explícito:
shouted = [f.upper() for f in fruits]

# también con diccionarios, y con filtro:
mapping = {
    f: f.upper()
    for f in fruits if not f.startswith("A")
}
```



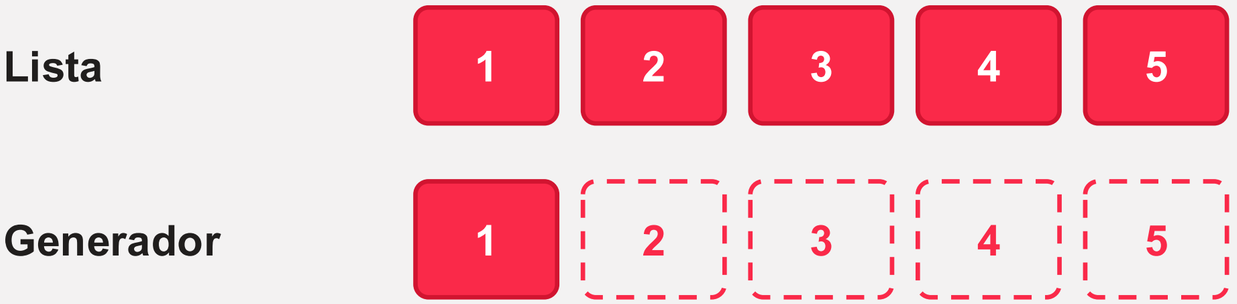
Generadores

- yield convierte una función en un generador: produce valores uno a la vez, sin guardar toda la secuencia en memoria
- Útil para secuencias muy largas o infinitas
- yield from delega en otro generador o iterable

```python
def number_generator():
    for i in range(1, 6):
        yield i  # "pausa" y entrega un valor

gen = number_generator()
next(gen)  # 1
next(gen)  # 2 (continúa donde se quedó)
```

Valores guardados en memoria



El generador guarda solo el valor actual y se "pausa" entre cada next().

Type hints y subclases

- Los type hints (str, Optional[str], Union[...]) documentan qué tipo se espera, sin forzarlo en tiempo de ejecución
- Una subclase hereda atributos y métodos de su clase padre y puede agregar o sobrescribir los propios

```python
from typing import Optional

class Libro:
    titulo: str
    autor: Optional[str] = None

class LibroDigital(Libro):
    # hereda de Libro y agrega un campo
    formato: str = "epub"
```



Decoradores

- Un decorador envuelve una función para agregarle comportamiento sin modificar su código interno
- @classmethod y @staticmethod son decoradores incluidos en Python
- Este patrón se usa para registrar herramientas de un agente (con @function_tool en algunos frameworks)

```python
def medir_tiempo(func):
    def envoltura(*args, **kwargs):
        inicio = time.time()
        resultado = func(*args, **kwargs)
        print(f"Tardó: {time.time()-inicio:.2f}s")
        return resultado
    return envoltura

@medir_tiempo
def tarea_lenta():
    ...
```



05

Programación Asíncrona

Corrutinas y await

- async def define una función asíncrona (una corrutina)
- Llamarla sin await solo crea el objeto corrutina — no ejecuta nada
- await la ejecuta y cede el control mientras espera (p. ej. una llamada de red)

```python
async def do_some_work():
    await asyncio.sleep(1)
    return "listo"

# no ejecuta nada, solo crea la corrutina:
do_some_work()

# así sí se ejecuta y se espera el resultado:
await do_some_work()
```

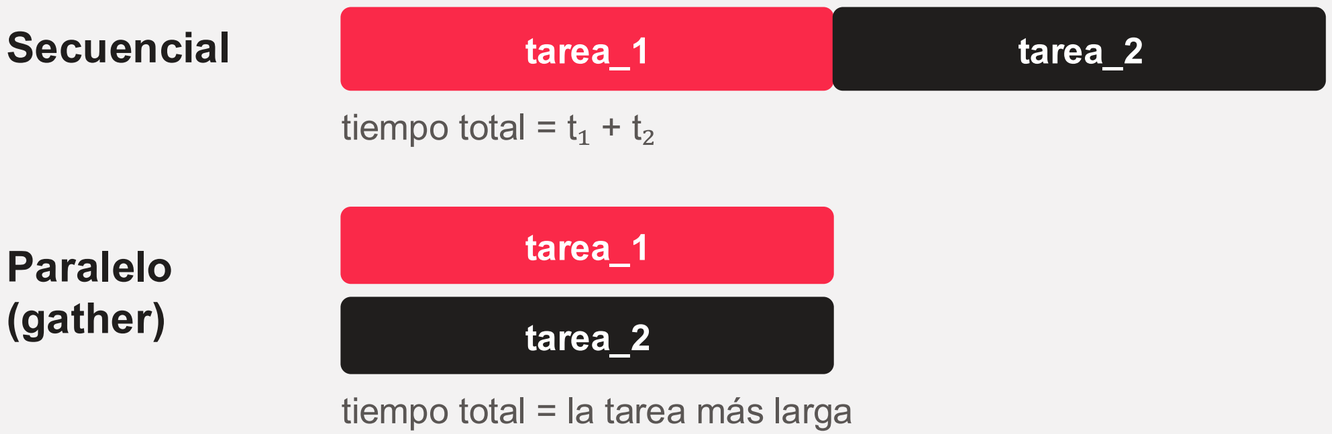


Secuencial vs. paralelo

- Encadenar await una tras otra sigue siendo secuencial — no gana nada en tiempo
- asyncio.gather() lanza varias corrutinas a la vez y espera a que todas terminen
- No es multithreading real: es concurrencia en un solo hilo, aprovechando los tiempos de espera

```python
# secuencial: espera una tarea, luego la otra
await tarea_1()
await tarea_2()

# en paralelo: ambas corren a la vez
await asyncio.gather(tarea_1(), tarea_2())
```



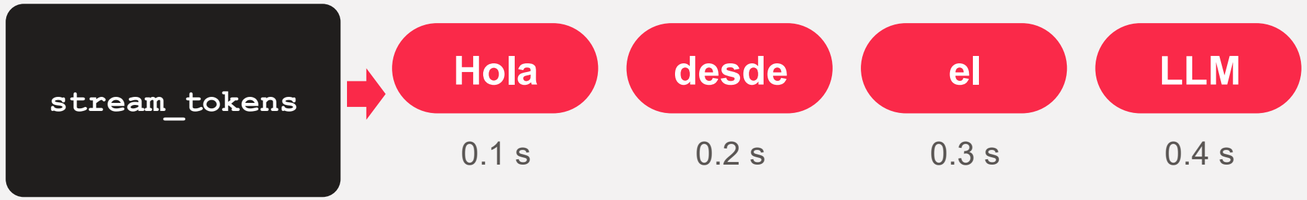
Generadores asíncronos: simulando streaming

- Combina async def con yield: produce valores uno a la vez, de forma asíncrona
- Así es como, en la práctica, un LLM "transmite" su respuesta token por token

```
async def stream_tokens(texto):
    for palabra in texto.split():
        await asyncio.sleep(0.1)
        yield palabra

async for token in stream_tokens("Hola desde el LLM"):
    print(token)
```

Los tokens llegan uno a uno



Cada token se muestra apenas se produce, sin esperar la respuesta completa.

06

Pydantic:
Validación de Datos

06

¿Qué es Pydantic y cuándo usarlo?

Pydantic define modelos de datos con tipos, y valida automáticamente que los datos recibidos los cumplan.



Cuándo usarlo

`API`

Validar datos de entrada de una API

`Config`

Definir configuración de una aplicación

`LLM`

Estructurar y limpiar la salida de un LLM

Definir, validar y serializar

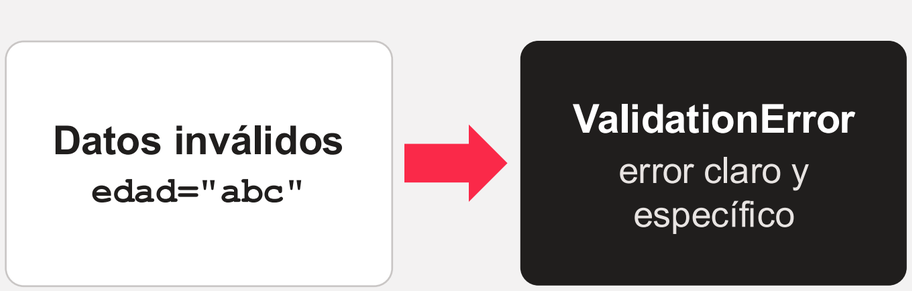
- BaseModel define la forma esperada de los datos con anotaciones de tipo normales de Python
- Si los datos no cumplen el modelo, Pydantic lanza un ValidationError claro y específico
- model_dump() / model_dump_json() serializan el objeto de vuelta a dict o JSON

```python
from pydantic import BaseModel

class Usuario(BaseModel):
    nombre: str
    edad: int

u = Usuario(nombre="Ana", edad="21")
# "21" se convierte a int

u.model_dump()        # -> dict de Python
u.model_dump_json()   # -> string JSON
```





Validación avanzada

- Field() agrega restricciones (gt, min_length, …) y metadatos a un campo
- @field_validator define una regla de validación personalizada para un campo específico
- model_config controla el comportamiento general del modelo (p. ej. si acepta campos extra)

```python
from pydantic import BaseModel, Field, field_validator

class Producto(BaseModel):
    precio: float = Field(gt=0)   # debe ser mayor que 0

    @field_validator("precio")
    def redondear(cls, v):
        return round(v, 2)
```

Qué pasa con un precio



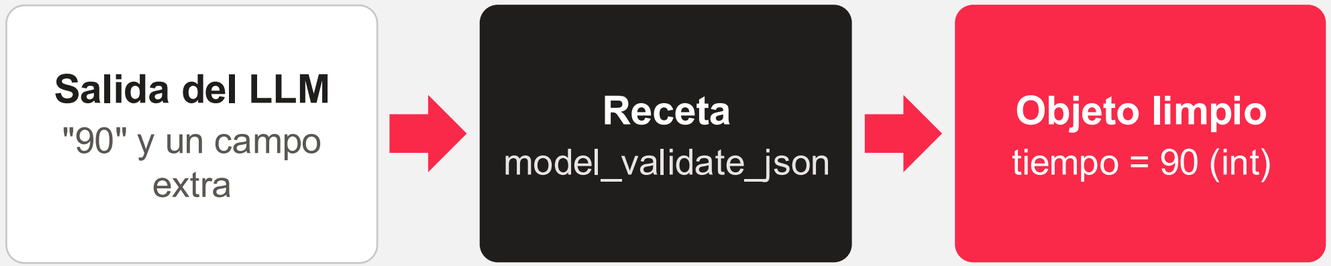
Caso práctico: limpiar la salida de un LLM

- La salida de un LLM en JSON puede traer tipos incorrectos ("90" en vez de 90) o campos que sobran
- Pydantic convierte "90" a int automáticamente e ignora el campo extra, devolviendo un objeto listo para usar

```python
class Ingrediente(BaseModel):
    nombre: str
    cantidad: float
    unidad: str

class Receta(BaseModel):
    nombre_receta: str
    ingredientes: List[Ingrediente]
    tiempo_preparacion_min: int

receta = Receta.model_validate_json(salida_llm)
```



RECURSOS

Resumen de la Unidad 1

	Tema	Notebook del curso
01	**IA, herramientas e instalación**	`Unidad 1/1_Introducción.ipynb`
02	**Notebooks en VS Code / Jupyter**	`Unidad 1/fundamentos/1_notebooks.ipynb`
03	**Python básico**	`Unidad 1/fundamentos/2_python_basics.ipynb`
04	**Python intermedio**	`Unidad 1/fundamentos/3_python_intermedio.ipynb`
05	**Programación asíncrona**	`Unidad 1/fundamentos/4_async_python.ipynb`
06	**Validación de datos con Pydantic**	`Unidad 1/fundamentos/5_pydantic.ipynb`